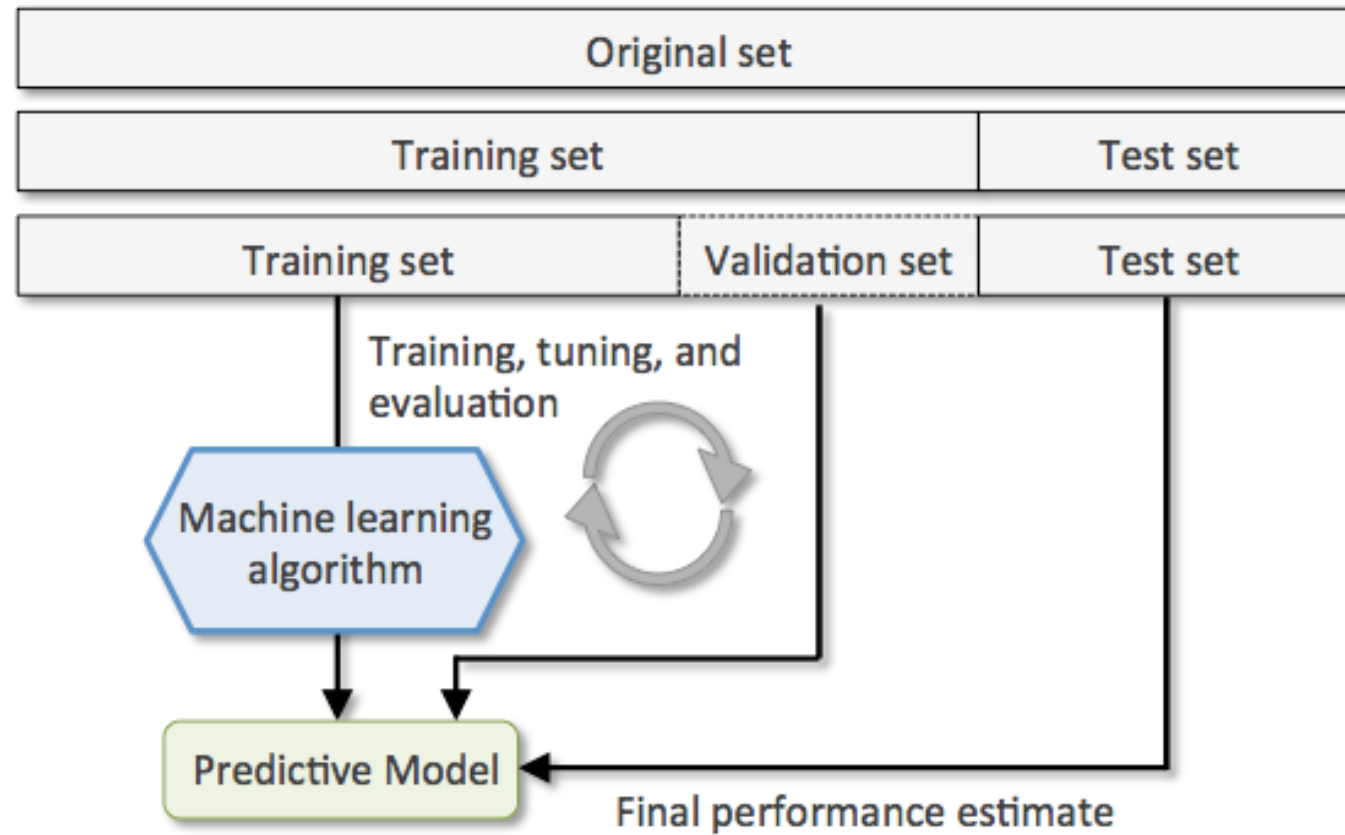


Holdout method and Regularization

Prof. Giuseppe Serra

Università degli Studi di Udine

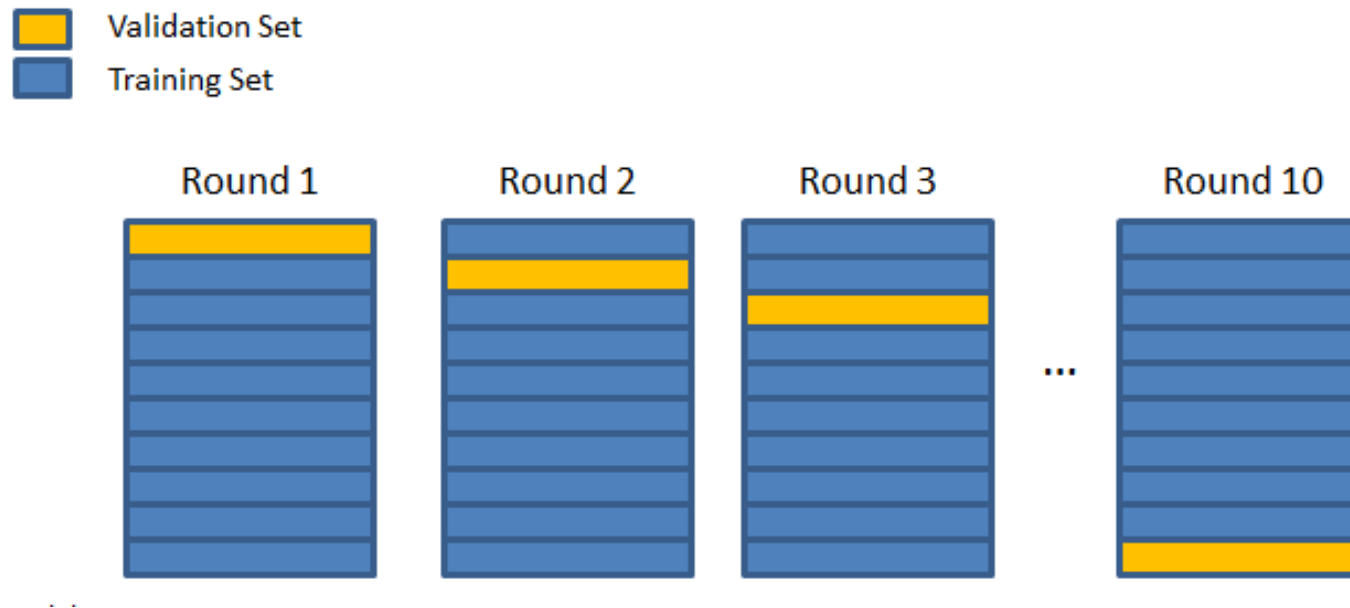
The holdout method



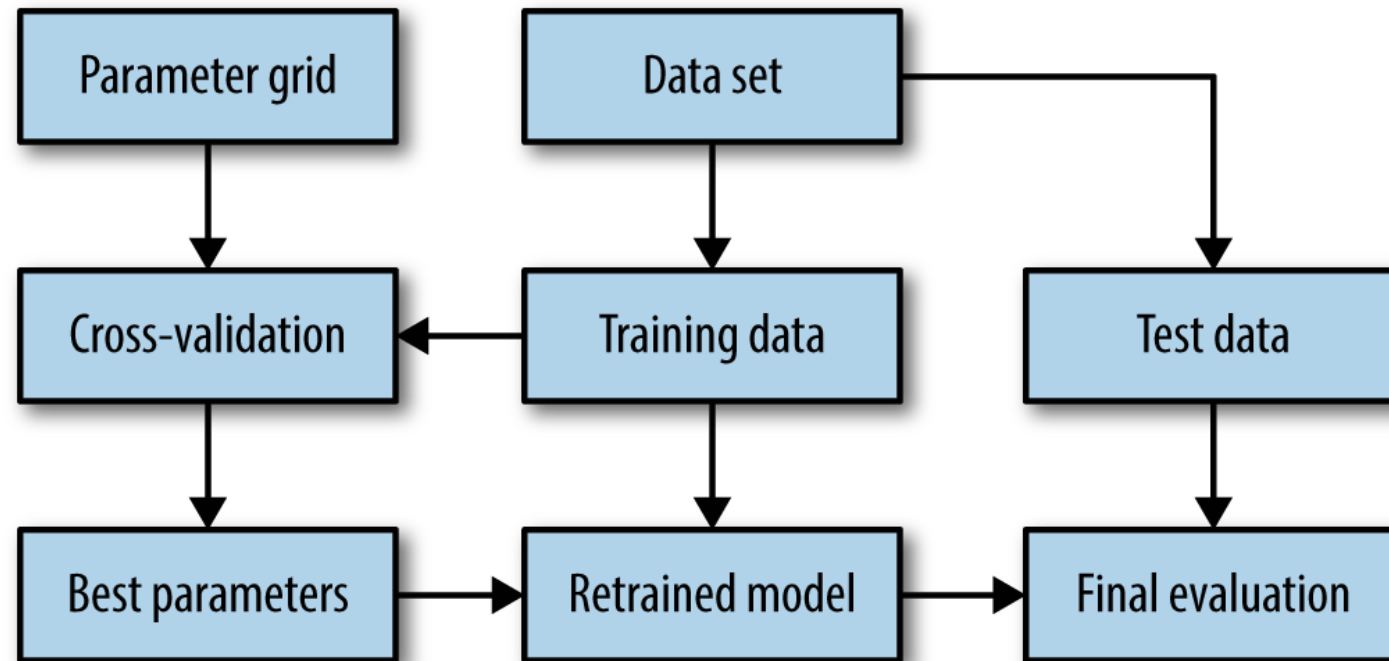
The holdout method: K-Fold Cross Validation

In K Fold cross validation, the data is divided into k subsets (10 in the figure below). Now the holdout method is repeated k times, such that each time, one of the k subsets is used as the validation set and the other $k-1$ subsets are put together to form a training set.

It is commonly used in the training, tuning and evaluation procedure (see slide before)

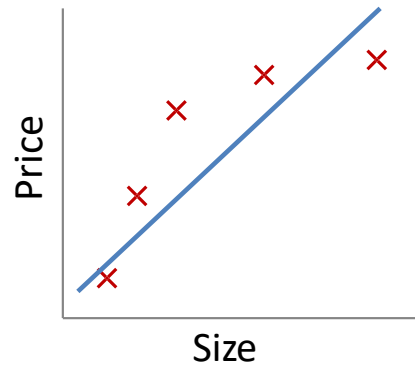


The holdout method



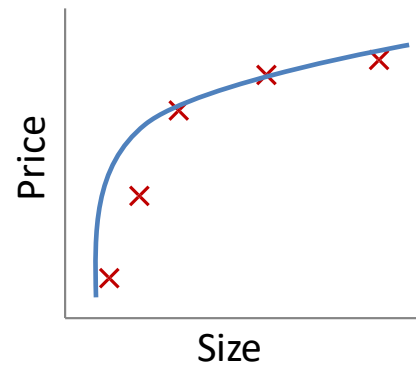
Regularization

Regression (2D-case)



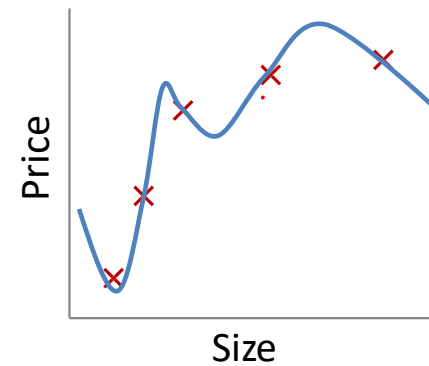
$$h_{\theta}(x) = \theta_0 + \theta_1 x$$

Underfit



$$h_{\theta}(x) = \theta_0 + \theta_1 x + \theta_2 x^2$$

Good

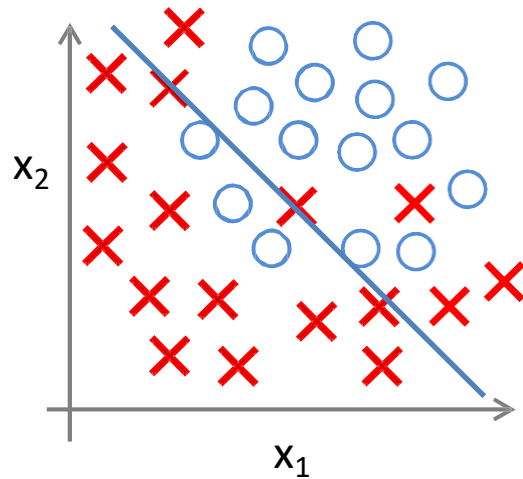


$$h_{\theta}(x) = \theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \theta_4 x^4$$

Overfit

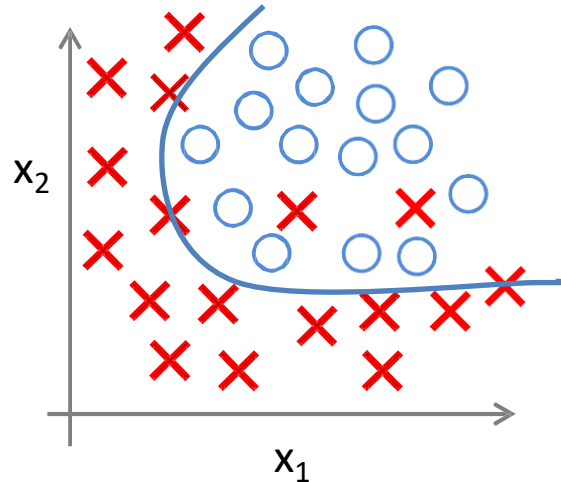
Overfitting: If we have too many features, the learned hypothesis may fit the training set very well, but fail to generalize to new examples (predict prices on new examples).

Classification (2D-case)



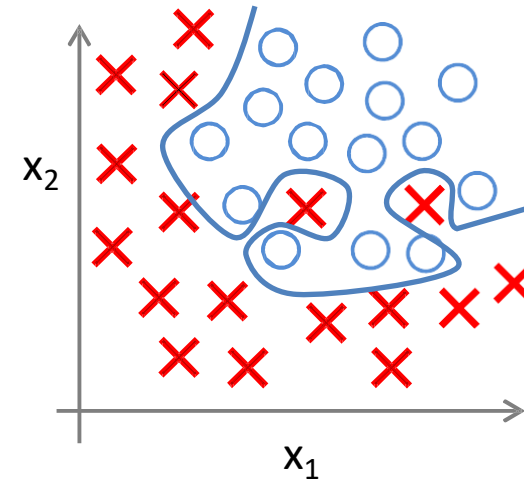
$$h_{\theta}(x) = g(\theta_0 + \theta_1 x_1 + \theta_2 x_2)$$

Underfit
(High Bias)



$$h_{\theta}(x) = g(\theta_0 + \theta_1 x + \theta_2 x_2 + \theta_3 x_1^2 + \theta_4 x_2^2 + \theta_5 x_1 x_2)$$

Good



$$h_{\theta}(x) = g(\theta_0 + \theta_1 x + \theta_2 x_2 + \theta_3 x_1^2 x_2 + \theta_4 x_1^2 x_2^2 + \theta_5 x_1^2 x_2^3 + \theta_6 x_1^3 x_2 + \dots)$$

Overfit
(High Variance)

Addressing overfitting (Multidimensional case)

- $x_1 = \text{size of house}$
- $x_2 = \text{n. of bedrooms}$
- $x_3 = \text{n. of floors}$
- $x_4 = \text{age of house}$
- $\vdots$
- x_{100}

In high dimensional space it is not very easy to determine which features have high correlation

Addressing overfitting (Multidimensional case)

Option

1. Reduce number of features

- Manually select which features to keep
- Model selection algorithm (PCA)

2. Regularization

- Keep all the features, but reduce magnitude/values of parameter θ_j
- Work well when we have a lot of features, each of which contributes a bit to prediction y

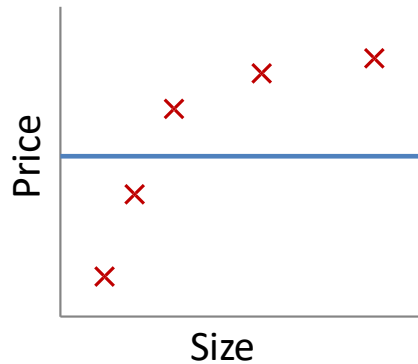
Regularization: Ridge regression

Minimization problem:

$$J(\theta) = \frac{1}{2m} \left[\sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 + \lambda \sum_{j=1}^n \theta_j^2 \right]$$

$$\min_{(\theta)} J(\theta)$$

Let's see an extreme case $\lambda = 10^{10}$ for our house prediction problem



Suppose we have four features ($n = 4$)
the result of the minimization

$$\theta_1 \approx \theta_2 \approx \theta_3 \approx \theta_4 \approx 0$$

so

$$h_{\theta}(x) = \theta_0$$

Bias/variance as function of the regularization parameter λ

Minimization problem of $J(\theta)$:

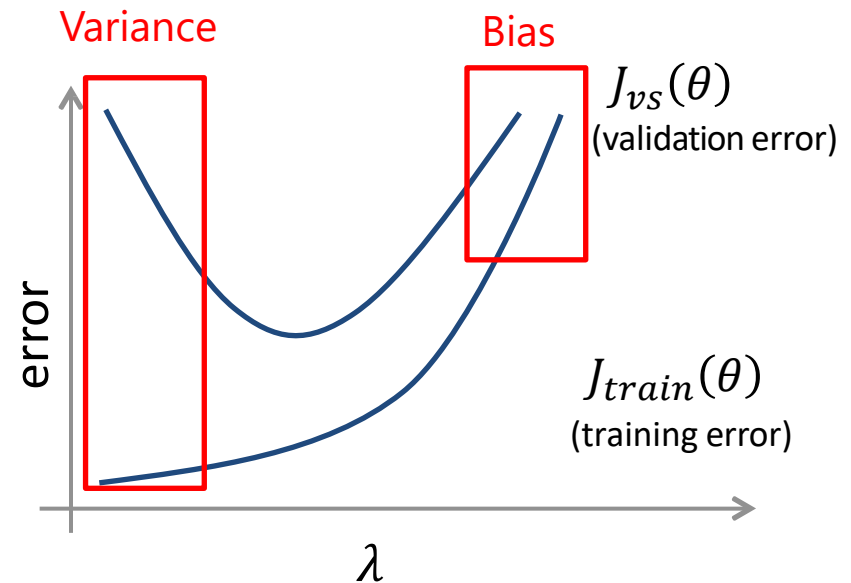
$$J(\theta) = \frac{1}{2m} \left[\sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 + \lambda \sum_{j=1}^n \theta_j^2 \right] = \frac{1}{2m} \left[\sum_{i=1}^m \text{Cost}(h_{\theta}(x), y) + \lambda \sum_{j=1}^n \theta_j^2 \right]$$

Where $\text{Cost}(h_{\theta}(x), y) = (h_{\theta}(x^{(i)}) - y^{(i)})^2$

After the minimization process we can compute:

$$J_{\text{train}}(\theta) = \frac{1}{m_{\text{train}}} \left[\sum_{i=1}^{m_{\text{train}}} \text{Cost}(h_{\theta}(x), y) \right]$$

$$J_{\text{vs}}(\theta) = \frac{1}{m_{\text{vs}}} \left[\sum_{i=1}^{m_{\text{vs}}} \text{Cost}(h_{\theta}(x), y) \right]$$



Regularization: Logistic regression

Minimization problem:

$$J(\theta) = \frac{1}{m} \left[\sum_{i=1}^m \text{Cost}(h_{\theta}(x), y) + \frac{\lambda}{2} \sum_{j=1}^n \theta_j^2 \right]$$

$$\min_{(\theta)} J(\theta)$$

Instead of using λ , some implementations (e.g. scikit-learn) use $C = \frac{1}{\lambda}$

So, the cost function can be written as:

$$J(\theta) = \frac{1}{m} \left[C \sum_{i=1}^m \text{Cost}(h_{\theta}(x), y) + \frac{1}{2} \sum_{j=1}^n \theta_j^2 \right]$$

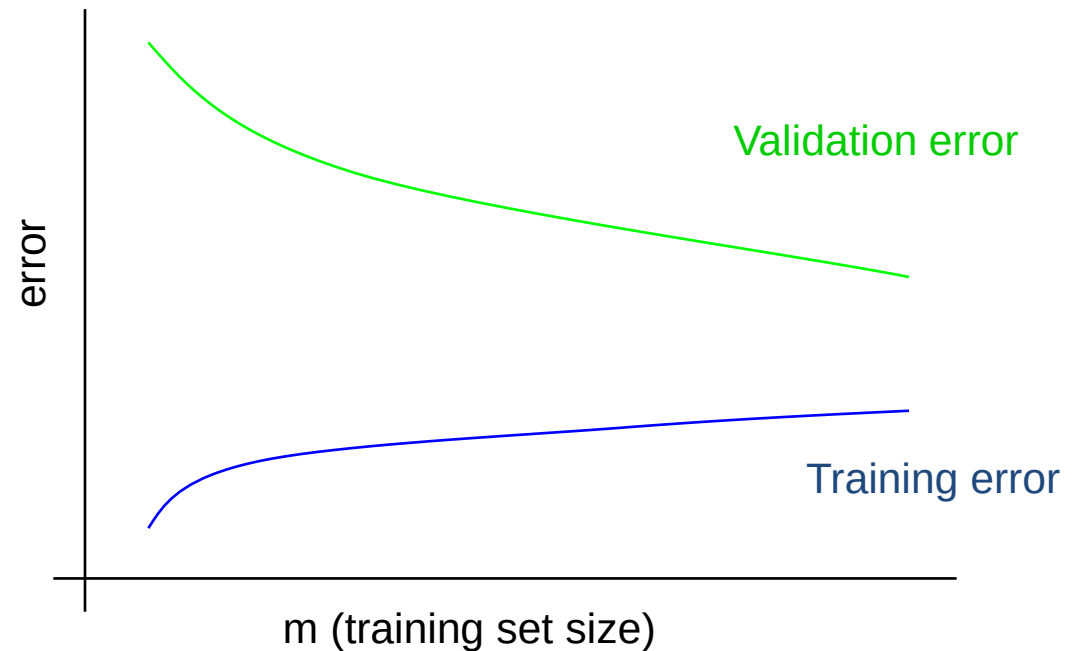
Diagnostic

Learning Curve:

Learning curves is often a very useful thing to plot. If either you wanted to sanity check that your algorithm is working correctly, or if you want to improve the performance of the algorithm.

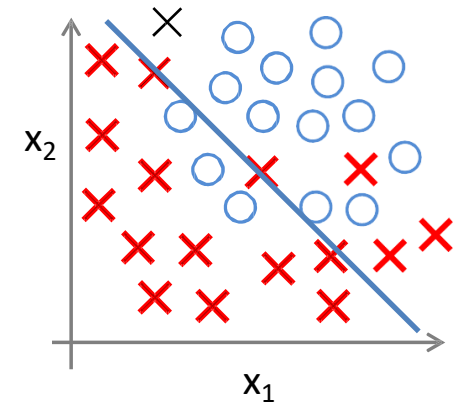
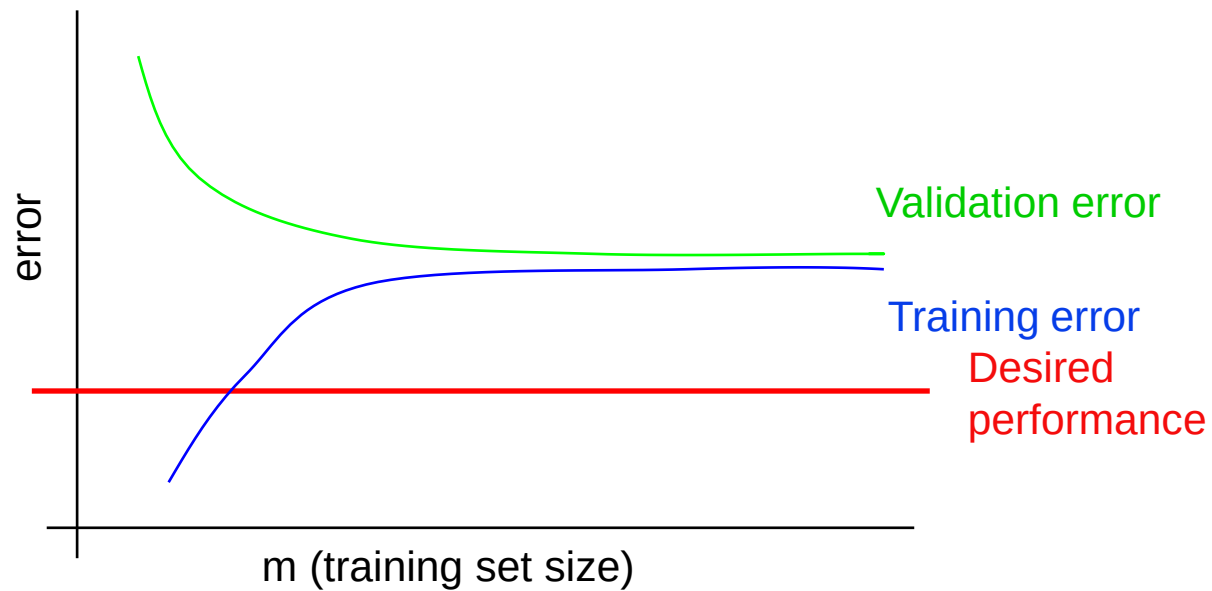
$$J_{train}(\theta) = \frac{1}{m_{train}} \left[\sum_{i=1}^{m_{train}} \text{Cost}(h_{\theta}(x), y) \right]$$

$$J_{vs}(\theta) = \frac{1}{m_{vs}} \left[\sum_{i=1}^{m_{vs}} \text{Cost}(h_{\theta}(x), y) \right]$$



Learning Curve: High Bias

Typical learning curve for high bias:

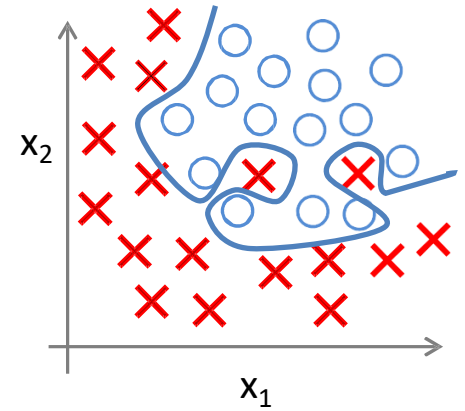
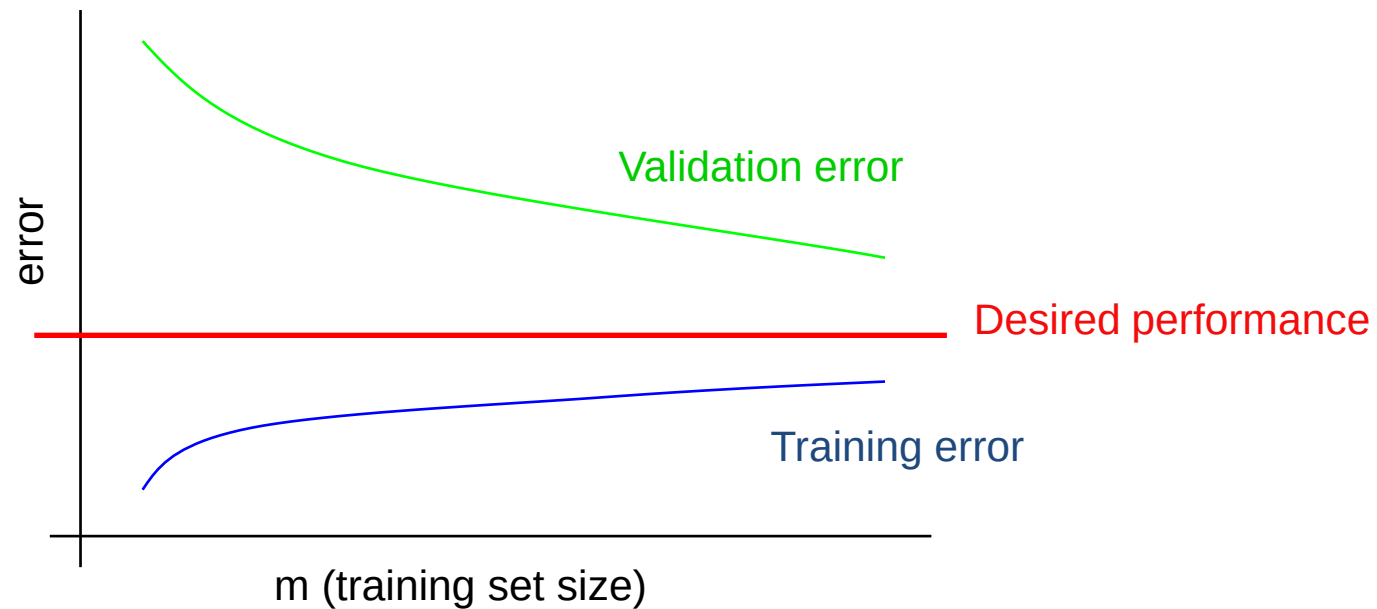


Small gap between training and validation error.

Even training error is unacceptably high. Suggests larger training set will not help.

Learning Curve: High Variance

Typical learning curve for high variance:



Large gap between training and validation error (overfitting effect)

Validation error still decreasing as m increase. Suggests larger training set will help.

Actions

Suppose you have implemented regularized logistic regression to predict patients. However, when you test your hypothesis in a new set of sample, you find that it makes unacceptably large errors in its prediction. What should you try next?

Fixes to try:

- Try getting more training examples. Fixes high variance.
- Try a smaller set of features. Fixes high variance.
- Try a larger set of features. Fixes high bias.
- Try decreasing λ . Fixes high bias.
- Try increasing λ . Fixes high variance.